Idea 46/50 - mutual_information_grokking


```
"Name": "mutual_information_grokking",
"Title": "Mutual Information Dynamics in Grokking: Tracing Information Flow
During Algorithmic Learning",
"Experiment": "1. Modify Transformer class to output representations from
input embedding, middle layer, and final layer. 2. Implement MINE (Mutual
Information Neural Estimation) for efficient mutual information
approximation. 3. Update training loop to compute and store mutual
information estimates between input-middle, input-output, and middle-output
every 500 steps. 4. Run experiments across all datasets, tracking mutual
information alongside existing performance metrics. 5. Plot the evolution
of mutual information alongside grokking curves and generalization gap. 6.
Analyze how mutual information changes before, during, and after grokking,
particularly in relation to the generalization gap. 7. Compare mutual
information dynamics between operations that grok quickly vs. slowly. 8.
Investigate correlations between changes in mutual information and grokking
speed or generalization performance.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 8,
"novel": true
```


Idea 47/50 - lottery_tickets_grokking


```
"Name": "lottery_tickets_grokking",
"Title": "Lottery Tickets in Grokking: Sparse Subnetworks and Sudden
Generalization",
"Experiment": "1. Implement iterative magnitude pruning for the Transformer
model. 2. Modify training loop for train-prune-reset cycles. 3. For each
dataset, run experiments with pruning levels of 50%
iterations. 4. Track metrics: time to grokking, final validation accuracy,
training loss, and 'grokking efficiency' (ratio of time to grokking for
sparse vs. dense network). 5. Plot learning curves for each pruning level,
highlighting grokking points. 6. Compare sparse network structures that
achieve grokking across operations. 7. Analyze correlation between pruning
level and grokking efficiency. 8. Implement simple MLP baseline without
grokking for comparison. 9. Visualize weight distributions of winning
tickets pre- and post-grokking.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 8,
"novel": false
```


Idea 48/50 - architecture_inductive_bias_grokking


```
"Name": "architecture_inductive_bias_grokking",
"Title": "Architectural Inductive Biases and Grokking: Comparing Sudden
Generalization Across Neural Network Types",
"Experiment": "1. Implement simplified 1D CNN and LSTM model classes
compatible with existing sequence-based datasets. 2. Modify training loop
to support multiple model types. 3. Run experiments comparing Transformer,
1D CNN, and LSTM models across modular arithmetic datasets. 4. Track
metrics: time to grokking, final validation accuracy, training loss, and
```

```
architecture-specific indicators (attention patterns for Transformer,
filter activations for CNN, forget gate activations for LSTM). 5. Plot
learning curves for each architecture, highlighting grokking points. 6.
Analyze how different architectures affect grokking behavior, speed, and
final performance for each operation type. 7. Compare internal
representations (using t-SNE) across architectures at key stages: pre-
grokking, during grokking transition, and post-grokking. 8. Investigate the
relationship between architectural inductive biases and the trade-off
between memorization and generalization in modular arithmetic tasks.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 8,
"novel": true
```

Idea 49/50 - shortcut_learning_grokking

```
"Name": "shortcut_learning_grokking",
"Title": "Shortcut Learning and Grokking: The Interplay Between Surface
Patterns and Deep Understanding in Algorithmic Learning",
"Experiment": "1. Modify AbstractDataset to include operation-specific
shortcuts: for modular arithmetic, make the result always even if the first
operand is even; for permutations, always swap the first two elements. 2.
Implement a function to gradually remove these shortcuts over training by
reducing their frequency. 3. Update the training loop to apply the shortcut
removal function. 4. Add a 'shortcut reliance' metric: the accuracy
difference between shortcut-following and shortcut-violating examples. 5.
Run experiments with varying shortcut removal rates across datasets. 6.
Track metrics: time to grokking, final validation accuracy, shortcut
reliance over time, and performance on a shortcut-free test set. 7. Plot
learning curves and shortcut reliance alongside grokking curves. 8. Analyze
how shortcut presence and removal affect grokking timing and quality. 9.
Compare attention patterns between models trained with and without
shortcuts at key stages.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 9,
"novel": true
```

Idea 50/50 - grokking_forgetting_complexity

```
"Name": "grokking_forgetting_complexity",
"Title": "Grokking and Forgetting: The Interplay of Task Complexity and
Sudden Generalization in Algorithmic Learning",
"Experiment": "1. Modify ModSumDataset to support multiple complexity
levels (e.g., modular addition with increasing prime moduli). 2. Update the
training loop to gradually introduce higher complexity levels while
continuously evaluating on all levels. 3. Implement a 'multi-complexity
evaluation' function to assess performance across all complexity levels
simultaneously. 4. Track metrics: time to grokking for each complexity
level, performance on lower complexity levels when grokking occurs on a
higher level, and a 'complexity forgetting score' (decrease in accuracy on
lower complexity levels). 5. Analyze the correlation between grokking
events and performance changes on other complexity levels. 6. Compare
internal representations (using cosine similarity of hidden states) across
```

```
complexity levels before and after grokking events. 7. Investigate trends
in grokking speed across increasing complexity levels. 8. Plot learning
curves for all complexity levels simultaneously, highlighting grokking
points and potential forgetting events. 9. Visualize the evolution of
representation similarities over time using heatmaps.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 9,
"novel": true
```